

SimpleFaceMe Implementation Plan

Revision History

- 2026-05-13: Initial implementation plan and requirements

Requirements Summary

Based on project decisions:

- **Target selection:** Closest target from LD2450 multi-target data
- **Servo range:** $\pm 90^\circ$ from center (no full-range calibration needed)
- **Update rate:** Configurable servo refresh period (build_flag), default 10Hz to match radar
- **Startup:** Servos at center position, disabled until first target detected
- **LD2450:** MUST use <https://github.com/RBEGamer/LD2450.git> library (custom driver FORBIDDEN)
- **Servo driver:** MUST use Adafruit PWM Servo Driver Library@^3.0.2 (custom driver FORBIDDEN)
- **Hardware pins:** UART1 (GPIO17/18) for LD2450, I2C (GPIO8/9) for PCA9685, all configurable via build_flags
- **Dual servo reference:** Use servo_validation project as baseline for tested servo configuration (20Kg servos)
- **Single servo:** MG996R is UNTESTED and requires pulse width calibration

Overview

This document outlines the implementation approach for the SimpleFaceMe demo. The goal is to create a working prototype quickly, prioritizing functionality over polish.

Implementation Plan - Detailed Task Breakdown

Task 1: PlatformIO Configuration & Build System

Files: `platformio.ini`, `include/config.h`

platformio.ini changes:

- Create `[env:single_servo]` environment
 - Platform: espressif32, Board: esp32-s3-devkitc1-n16r8
 - Build flags: NUM_SERVOS=1, SENSOR_OFFSET_X=0, SENSOR_OFFSET_Y=0, SERVO_OFFSET_X=0, SERVO_OFFSET_Y=-50
 - Build flags: LD2450_BAUD=256000, PCA9685_ADDR=0x40, SERVO_FREQ=50
 - Build flags: SERVO_UPDATE_PERIOD_MS=100, DEBUG_VERBOSE=0
 - Lib deps:
 - <https://github.com/RBEGamer/LD2450.git>
 - adafruit/Adafruit PWM Servo Driver Library@^3.0.2
 - Wire, SPI

- Monitor speed: 115200
- **Hardware:** MG996R servo (UNTESTED - needs calibration in Task 6)
- Create `[env:dual_servo]` environment
 - Same platform/board as single_servo
 - Build flags: NUM_SERVOS=2
 - Build flags: SERVO1_OFFSET_X=-150, SERVO1_OFFSET_Y=-100, SERVO2_OFFSET_X=150, SERVO2_OFFSET_Y=-100
 - Build flags: Same LD2450_BAUD, PCA9685_ADDR, SERVO_FREQ, SERVO_UPDATE_PERIOD_MS, DEBUG_VERBOSE
 - Same lib deps
 - **Hardware:** Two-head servo pair (TESTED - use servo_validation settings)

include/config.h creation:

- Add `#error` checks for all required build_flags: NUM_SERVOS, SENSOR_OFFSET_X/Y, LD2450_BAUD, PCA9685_ADDR, SERVO_FREQ, SERVO_UPDATE_PERIOD_MS
- Add conditional checks for SERVO_OFFSET_X/Y vs SERVO1/SERVO2_OFFSET_X/Y based on NUM_SERVOS
- Add categorized debug system with token concatenation:
 - Categories: SYSTEM, SENSOR, MOTOR, GEOMETRY
 - Each category independently enabled/disabled via build_flags
 - Compact macros: `SENSOR_PRINT(x)`, `MOTOR_PRINTLN(x)`, `GEOMETRY_PRINTF(...)` etc.
 - Uses preprocessor token pasting: `CAT_ENABLED(cat)` → `DEBUG_##cat`
 - Allows fine-grained debug control without code changes

Verification: Both environments compile successfully (even with stub main.cpp)

Task 2: LD2450 Library Integration

Files: `platformio.ini`, wrapper code in `src/main.cpp`

FORBIDDEN: Writing custom LD2450 driver - MUST use existing library

Library: <https://github.com/RBEGamer/LD2450.git> (as used in faceme_test)

Integration approach:

- Add library to lib_deps in platformio.ini
- Study library API to understand:
 - Initialization (serial port, baud rate)
 - Reading target data
 - Target structure/format
 - Getting closest target or iterating targets
- Create simple wrapper/helper functions in main.cpp if needed for:
 - Selecting closest target from multi-target data
 - Tracking time since last valid target (for timeout)
 - Extracting X,Y coordinates in format needed by geometry calculation

Reference: `/faceme_test/platformio.ini` shows library usage

Exit Criteria (MUST PASS):

1. Code compiles without errors
2. LD2450 library initializes successfully (no I/O errors)
3. **Successful reads from sensor** - Verify data packets received
4. Target coordinates (X, Y) successfully extracted
5. Debug output shows valid target data when object detected
6. Can identify when no targets present vs. valid target detected

Verification method:

- Enable `DEBUG_VERBOSE=1`
 - Monitor serial output
 - Wave hand in front of sensor
 - Confirm target coordinates printed and update in real-time
 - Verify "no target" state detected correctly
-

Task 3: Servo Control Implementation

Files: `src/servo_control.h`, `src/servo_control.cpp`

Library: `adafruit/Adafruit PWM Servo Driver Library@^3.0.2` (REQUIRED)

Hardware Configuration Handling:

Dual Servo (Two Heads) - TESTED CONFIGURATION:

- Use exact settings from servo_validation project
- PCA9685 I2C address: 0x40
- Servo frequency: 50Hz
- PWM pulse range: Known/calibrated from servo_validation testing
- Two 20Kg servos already validated
- Copy PWM calculation formulas from servo_validation/src/main.cpp

Single Servo (MG996R) - UNTESTED CONFIGURATION:

- PCA9685 I2C address: 0x40 (same hardware)
- Servo frequency: 50Hz
- PWM pulse range: Conservative defaults (500-2500µs)
- **NOTE:** MG996R NOT yet calibrated - may need adjustment
- **TESTING REQUIRED:** Verify $\pm 90^\circ$ range achievable, adjust pulse limits if needed
- Use same PCA9685 code but expect to tune pulse width values during Task 6

servo_control.h interface:

```
class ServoControl {
public:
    ServoControl(uint8_t i2c_address, uint8_t num_servos);
    void begin();
    void setPosition(uint8_t servo_num, float angle_degrees); // -90 to +90
    void setCenter(uint8_t servo_num);
    void centerAll();
private:
    Adafruit_PWMServoDriver pwm;
    uint8_t numServos;
    int angleToPulse(float angle); // Convert angle to PWM pulse width
};
```

servo_control.cpp implementation:

- Use Adafruit_PWMServoDriver library for all PCA9685 interaction
- Copy PWM calculation from servo_validation (tested for dual servo build)
- For MG996R: Start with conservative pulse range, document that tuning needed
- Reference `/servo_validation/src/main.cpp` lines for PCA9685 setup and PWM formulas

Verification: Compile check, note MG996R requires hardware testing in Task 6

Task 4: Geometry & Angle Calculation

Files: `src/geometry.h`, `src/geometry.cpp`

geometry.h interface:

```
struct Vector2D {
    float x;
    float y;
};

float calculatePanAngle(Vector2D target, Vector2D servoOffset);
```

geometry.cpp implementation:

- `calculatePanAngle(target, servoOffset)`:
 - Calculate relative position: $rel_x = target.x - servoOffset.x$, $rel_y = target.y - servoOffset.y$
 - Calculate angle: $angle = atan2(rel_x, rel_y) * 180.0 / PI$
 - Return angle in degrees
 - Note: $atan2(x, y)$ not $atan2(y, x)$ because 0° = facing forward (+Y axis)

Verification: Unit test with known values (e.g., target at (100, 100) with servo at (0, 0) should give 45°)

Task 5: Main Application Integration

Files: `src/main.cpp`

main.cpp structure:

```

#include <Arduino.h>
#include "config.h"
#include "ld2450.h"
#include "servo_control.h"
#include "geometry.h"

// Hardware instances
HardwareSerial radarSerial(1); // UART1 for LD2450
LD2450 radar(radarSerial, LD2450_BAUD);
ServoControl servos(PCA9685_ADDR, NUM_SERVOS);

// Timing
unsigned long lastServoUpdate = 0;

void setup() {
    Serial.begin(115200); // Debug output
    DEBUG_PRINT("SimpleFaceMe starting...\n");

    radar.begin();
    servos.begin(); // Centers servos

    DEBUG_PRINT("Initialization complete\n");
}

void loop() {
    // Update radar data
    if (radar.update()) {
        Target target = radar.getPrimaryTarget();

        if (target.valid && (millis() - lastServoUpdate >= SERVO_UPDATE_PERIOD_MS))
        {
            Vector2D targetPos = {(float)target.x, (float)target.y};

            #if NUM_SERVOS == 1
                Vector2D servoOffset = {SERVO_OFFSET_X, SERVO_OFFSET_Y};
                float angle = calculatePanAngle(targetPos, servoOffset);
                servos.setPosition(0, angle);
                DEBUG_PRINT("Target: "); DEBUG_PRINT(target.x); DEBUG_PRINT(",");
            DEBUG_PRINT(target.y);
                DEBUG_PRINT(" Angle: "); DEBUG_PRINT(angle); DEBUG_PRINT("\n");
            #else
                Vector2D servo1Offset = {SERVO1_OFFSET_X, SERVO1_OFFSET_Y};
                Vector2D servo2Offset = {SERVO2_OFFSET_X, SERVO2_OFFSET_Y};
                float angle1 = calculatePanAngle(targetPos, servo1Offset);
                float angle2 = calculatePanAngle(targetPos, servo2Offset);
                servos.setPosition(0, angle1);
                servos.setPosition(1, angle2);
                DEBUG_PRINT("Target: "); DEBUG_PRINT(target.x); DEBUG_PRINT(",");
            DEBUG_PRINT(target.y);
                DEBUG_PRINT(" Angles: "); DEBUG_PRINT(angle1); DEBUG_PRINT(",");
            DEBUG_PRINT(angle2); DEBUG_PRINT("\n");
            #endif
        }
    }
}

```

```

        lastServoUpdate = millis();
    }
}

// Timeout - return to center if no target
if (radar.timeSinceLastTarget() > 5000) { // 5 second timeout
    servos.centerAll();
    delay(100); // Don't spam center commands
}
}

```

Verification: Code compiles for both environments

Task 6: Testing & Refinement

Testing sequence - DUAL SERVO FIRST (known hardware):

1. Compile dual_servo environment - verify no errors
2. Upload dual_servo build
3. Connect LD2450 and two-head servo pair
4. Enable DEBUG_VERBOSE=1, monitor serial output
5. Verify target detection using LD2450 library
6. Check angle calculations for both servos
7. **Confirm servos track target** (should work - using servo_validation config)
8. Test timeout behavior (servos return to center)

Testing sequence - SINGLE SERVO (UNCALIBRATED MG996R):

1. Compile single_servo environment
2. Upload single_servo build
3. Connect LD2450 and MG996R servo
4. **CALIBRATION REQUIRED:**
 - Test center position (0°) - verify servo is actually centered
 - Test +90° and -90° positions
 - Observe if servo reaches limits or stalls
 - Adjust PWM pulse width constants if needed
 - May need to modify PULSE_MIN_US/PULSE_MAX_US values
5. Once calibrated, verify tracking behavior
6. Test timeout behavior

Expected refinements:

- MG996R: PWM pulse width tuning (likely different from dual servo values)
 - Both builds: SERVO_UPDATE_PERIOD_MS adjustment if jittery
 - Both builds: Basic filtering if radar coordinates jump
 - Verify $\pm 90^\circ$ limits don't cause mechanical binding
-

File Creation Order

1. `platformio.ini` - Environments and library dependencies
2. `include/config.h` - Configuration validation
3. `src/geometry.h` + `src/geometry.cpp` - Simple, no dependencies
4. `src/servo_control.h` + `src/servo_control.cpp` - Servo driver using Adafruit library
5. `src/main.cpp` - Integration using RBEGamer LD2450 library
6. Study RBEGamer/LD2450 library API and examples

Libraries Used

REQUIRED (cannot write custom implementations):

- LD2450: <https://github.com/RBEGamer/LD2450.git> (radar sensor)
- Adafruit PWM Servo Driver Library@^3.0.2 (PCA9685 control)

Hardware Configuration Summary

Dual Servo Build:

- Hardware: Two 20Kg servos from servo_validation project
- Status: ✓ TESTED and CONFIGURED
- Settings: Copy from servo_validation/src/main.cpp
- Expected outcome: Should work immediately

Single Servo Build:

- Hardware: MG996R servo
- Status: ⚠ UNTESTED - NO CALIBRATION
- Settings: Conservative defaults, expect to tune
- Expected outcome: Requires pulse width calibration in Task 6

Estimated Time

- Task 1 (Config): 15 min
- Task 2 (LD2450 Library Study): 30 min (learn API, not write driver)
- Task 3 (Servo): 30 min (using servo_validation + Adafruit library)
- Task 4 (Geometry): 15 min (simple math)
- Task 5 (Main): 45 min (integration with libraries)
- Task 6 (Test): 60 min (dual servo quick, MG996R calibration time)

Total: ~3.5 hours (extra time for MG996R calibration)

File Structure

```
SimpleFaceMe/
├── platformio.ini      # Two build environments
├── README.md           # Project overview
├── docs/
│   ├── architecture.md # System design
│   └── implementation.md # This file
├── include/
│   └── config.h        # Configuration validation
├── src/
│   ├── main.cpp        # Main loop and setup
│   ├── ld2450.cpp/h    # Radar driver
│   ├── servo_control.cpp/h # Servo angle calculation
│   └── geometry.cpp/h  # Coordinate transformations
└── lib/                # (empty, using platform libraries)
```

Code Architecture

Main Loop Structure

```
void loop() {  
    // 1. Read radar data  
    if (ld2450.update()) {  
        Target target = ld2450.getPrimaryTarget();  
        SENSOR_PRINTF("Target: (%d, %d)\n", target.x, target.y);  
  
        // 2. Calculate servo angles  
        #ifdef SINGLE_SERVO_BUILD  
            float angle = calculateServoAngle(target, servoOffset);  
            GEOMETRY_PRINTF("Angle: %.1f°\n", angle);  
            setServoPosition(0, angle);  
            MOTOR_PRINTF("Servo 0 → %.1f°\n", angle);  
        #endif  
  
        #ifdef DUAL_SERVO_BUILD  
            float angle1 = calculateServoAngle(target, servo1Offset);  
            float angle2 = calculateServoAngle(target, servo2Offset);  
            GEOMETRY_PRINTF("Angles: %.1f°, %.1f°\n", angle1, angle2);  
            setServoPosition(0, angle1);  
            setServoPosition(1, angle2);  
            MOTOR_PRINTF("Servos: 0→%.1f° 1→%.1f°\n", angle1, angle2);  
        #endif  
    }  
  
    // 3. Handle timeout  
    if (ld2450.timeSinceLastTarget() > TIMEOUT_MS) {  
        SYSTEM_PRINTLN("Timeout - returning to center");  
        returnToCenter();  
    }  
}
```

Configuration Pattern (per platformio-practices)

All parameters defined in platformio.ini:

```

build_flags =
  -D NUM_SERVOS=1
  -D SENSOR_OFFSET_X=0
  -D SENSOR_OFFSET_Y=0
  -D SERVO_OFFSET_X=0
  -D SERVO_OFFSET_Y=-50
  -D LD2450_BAUD=256000
  -D PCA9685_ADDR=0x40
  -D SERVO_FREQ=50
  -D SERVO_UPDATE_PERIOD_MS=100 ; 10Hz default, configurable
  -D DEBUG_VERBOSE=0
  ; Debug categories (independently enabled)
  -D DEBUG_SYSTEM=1
  -D DEBUG_SENSOR=0
  -D DEBUG_MOTOR=0
  -D DEBUG_GEOMETRY=0

```

With validation in config.h:

```

#ifndef NUM_SERVOS
  #error "NUM_SERVOS must be defined"
#endif
#ifndef SENSOR_OFFSET_X
  #error "SENSOR_OFFSET_X must be defined"
#endif
// ... etc

```

Libraries & Dependencies

- **Adafruit PWM Servo Driver Library** (~3.0.2) - PCA9685 control
- **Wire** - I2C communication
- **Arduino.h** - Framework

Testing Strategy

Since this is a quick demo, testing will be minimal:

1. **Unit:** Verify angle calculations with known coordinates
2. **Integration:** Test with simulated radar data (hardcoded targets)
3. **Hardware:** Final validation with actual LD2450 and servos

Known Limitations (Quick & Dirty)

- No servo calibration (assumes default MG996R range)
- Basic target selection (closest or first)
- Minimal filtering (may be jittery)

- No multi-threading or RTOS (simple sequential loop)
- Hardcoded protocol (single target tracking mode)

Future Enhancements (Out of Scope)

- Servo calibration routine
- Smooth motion interpolation
- Multiple target handling
- Web interface for configuration
- Persistent settings in EEPROM/SPIFFS